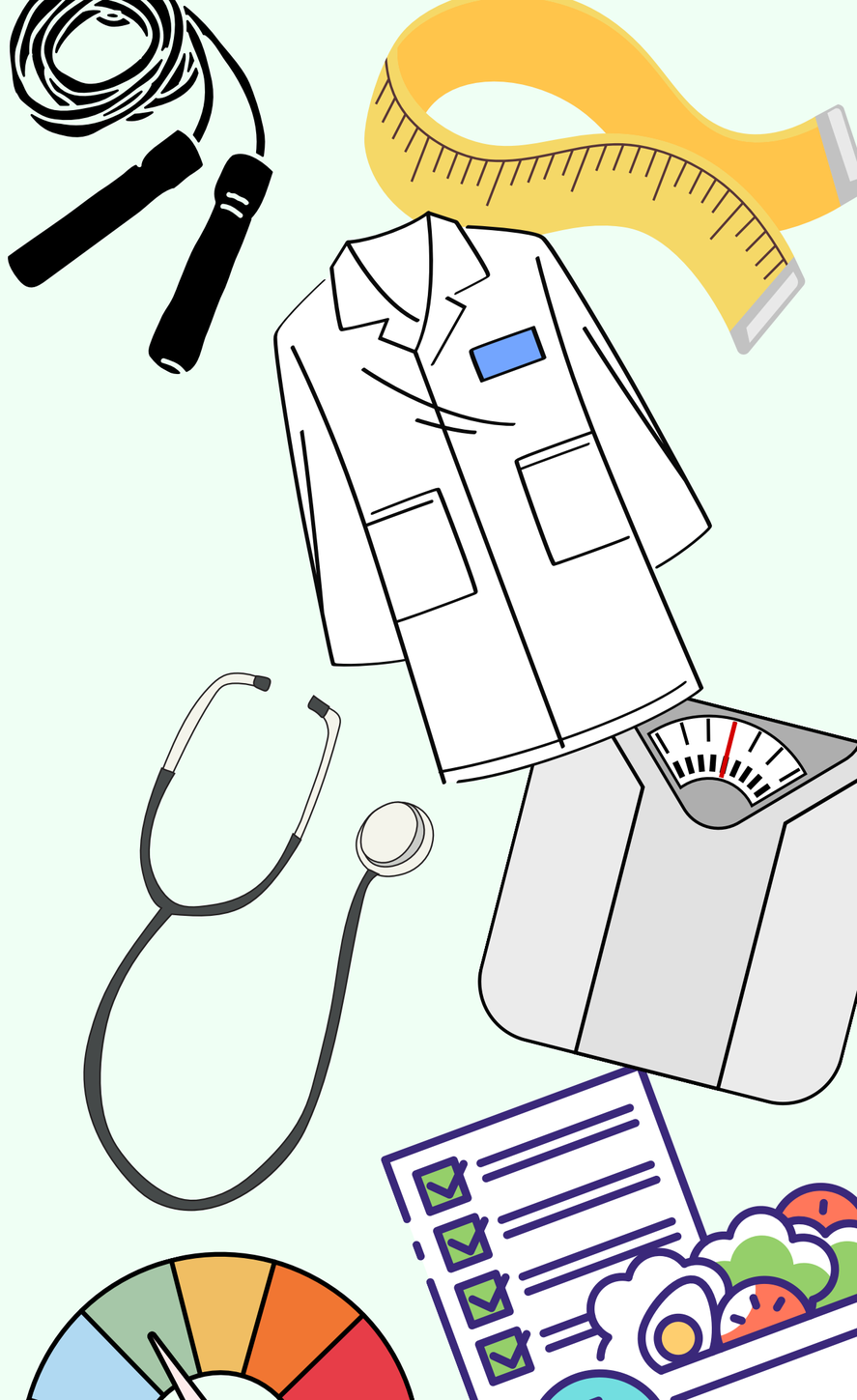
Bizihotz

Iñigo Fiat Sarasola



¿Qué es Bizihotz?

Bizihotz es una aplicación software y una plataforma web destinada a los profesionales de un centro de nutrición privado. La herramienta permitirá gestionar las citas con pacientes, realizar seguimientos de dietas personalizadas y mejorar la comunicación entre nutricionistas, médicos y otros especialistas.



Objetivos

Centralizar

- Unificar en una única plataforma todos los procesos de atención nutricional.
- Mejorar la eficiencia operativa

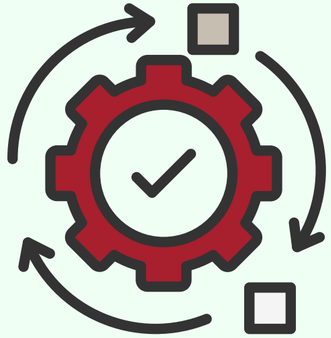
Personalizar

- Permitir la creación de planes nutricionales altamente personalizados
- Facilitar el seguimiento continuo y detallado de la evolución del paciente
- Proporcionar herramientas de análisis para evaluar la adherencia y efectividad

Mejorar

- Establecer canales seguros y eficientes de comunicación entre especialistas
- Facilitar la discusión de casos complejos y el aprendizaje colaborativo

Funcionalidades



01

Gestión de citas y seguimiento de pacientes

- Agendar citas para consultas nutricionales, seguimientos de dietas y evaluaciones periódicas.
- Crear y modificar planes alimenticios personalizados para cada paciente.
- Seguimiento de la evolución del paciente (peso, medidas, hábitos alimenticios, etc.).

02

Comunicación interna

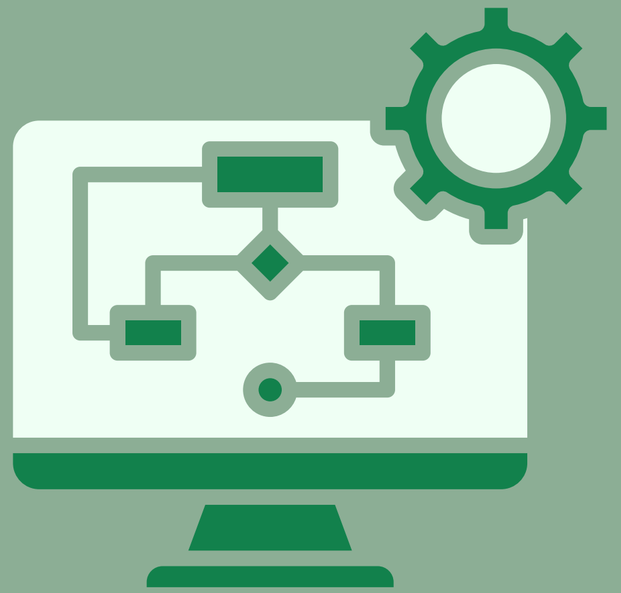
- Mensajería instantánea para resolver dudas entre nutricionistas y otros especialistas

03

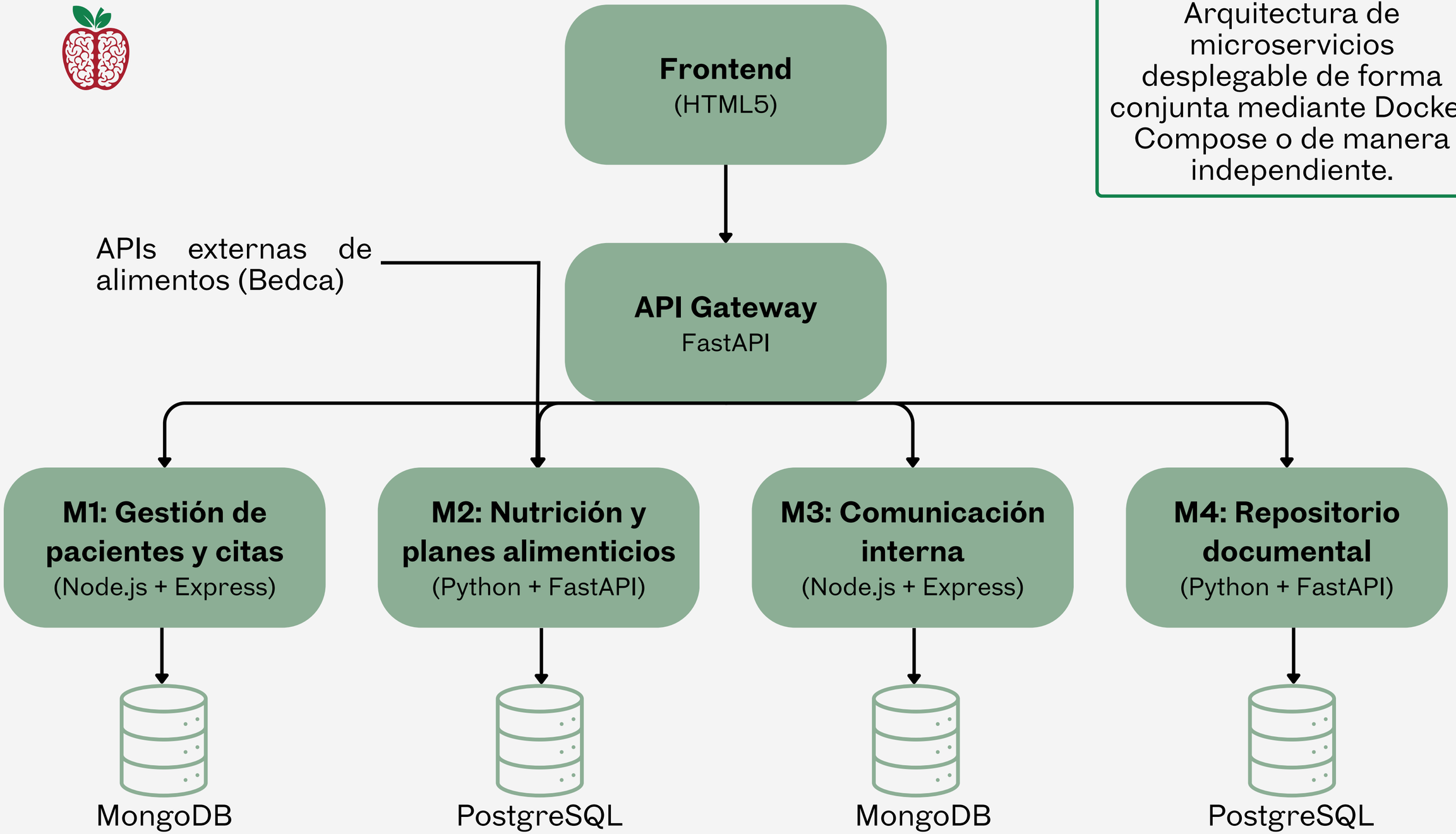
Repositorio centralizado de documentación nutricional

- Acceso a guías nutricionales, estudios de investigación y protocolos dietéticos.
- Recetas saludables y fáciles de adaptar a diferentes planes alimenticios.

Arquitectura



Arquitectura de microservicios desplegable de forma conjunta mediante Docker Compose o de manera independiente.



M1: Gestión de pacientes y citas

(Node.js + Express)

1. API RESTful para crear, leer, modificar y eliminar citas pacientes
2. Gestión de calendarios y citas
3. Registro de evolución y métrica de pacientes
4. Base de Datos: MongoDB
5. Se consume vía API Gateway
6. JWT



M2: Nutrición y planes alimenticios

(Python + FastAPI)

1. API RESTful para crear y gestionar dietas
2. Cálculos nutricionales
3. Integración con APIs externas de datos nutricionales
4. Base de Datos: PostgreSQL (integridad referencial para recetas y alimentos)



M3: Comunicación interna

(Node.js + Express)

1. API RESTful para manejar la comunicación entre diferentes profesionales
2. Comunicación en tiempo real
3. Base de Datos: MongoDB
4. Socket.IO



M4: Repositorio documental

(Python + FastAPI)

1. API RESTful para gestión de documentos
2. Sistema de búsqueda avanzada
3. Base de datos: PostgreSQL
4. Integración vía Gateway



API Gateway

(FastAPI)

1. FastAPI + Uvicorn
2. Punto de entrada único
3. Enrutamiento a microservicios
4. Documentación OpenAPI



Breve resumen

El API Gateway, desarrollado en FastAPI, actúa como punto de acceso único al sistema, encargándose del enrutamiento de las peticiones hacia los diferentes microservicios.

El frontend (HTML5) se comunica con el Gateway mediante APIs RESTful, garantizando una integración limpia y escalable.

Además, se integran fuentes externas de datos nutricionales para enriquecer la información almacenada.



Herramientas a utilizar



Backend

Microservicios Node.js

- Runtime: Node.js
- Framework: Express.js
- WebSockets: Socket.io (para mensajería en tiempo real)

Microservicios Python

- Framework: FastAPI
- ORM: PostgreSQL

API Gateway

- Framework: Express.js
- Documentación: Swagger UI + OpenAPI 3.0

Bases de Datos

MongoDB

- Datos de pacientes
- Mensajes
- Citas

PostgreSQL

- Recetas
- Alimentos
- Dietas
- Documentación

Frontend

Framework

- HTML5, CSS y JavaScript

DevOps

Versiones

- Git / Github

Contenedorización

- Docker / Docker Compose

Breve resumen

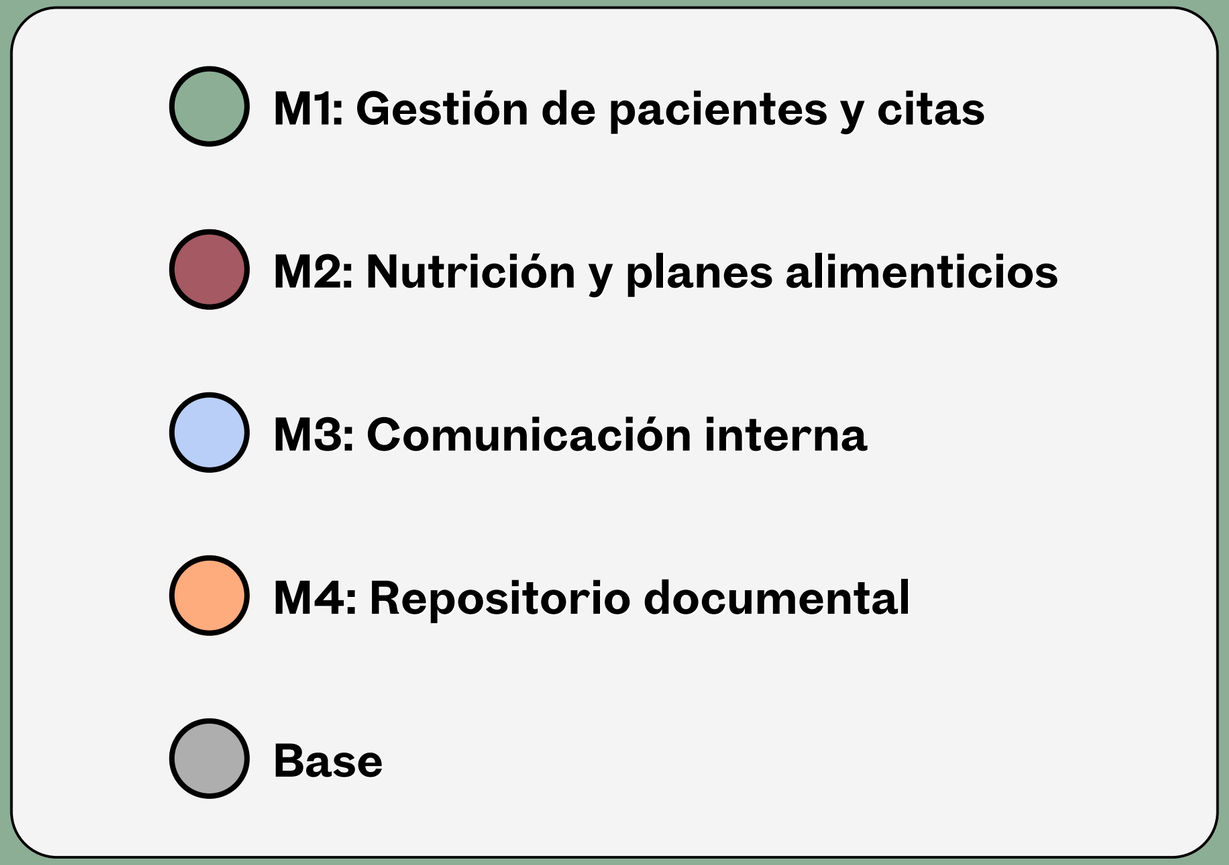
En el backend, se utilizan Node.js + Express y Python + FastAPI, distribuidos en microservicios independientes que exponen APIs RESTful. Cada uno de ellos se comunica con el API Gateway, encargado de centralizar las peticiones.

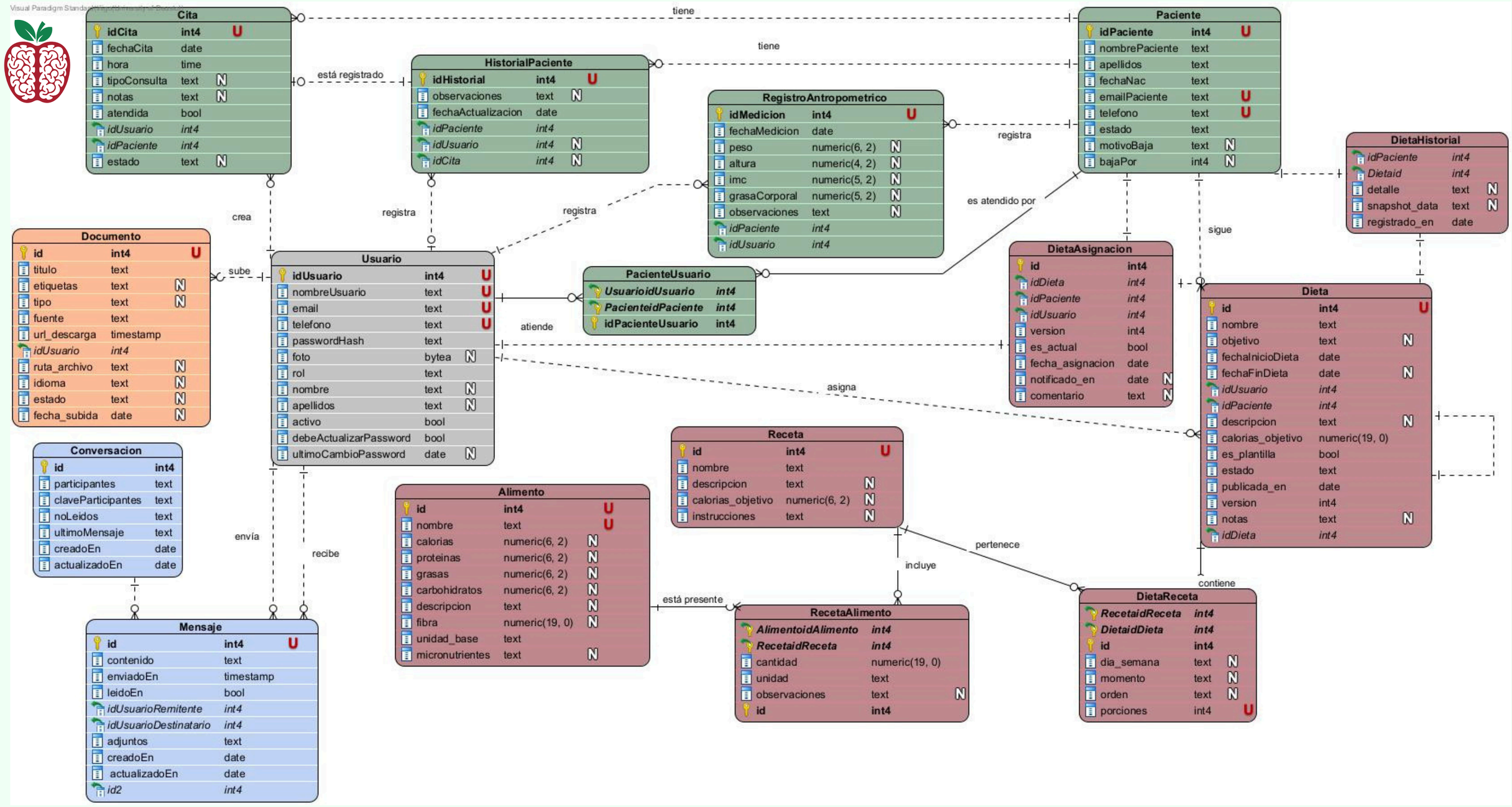
En el frontend, se va a hacer uso de HTML5.

Para la persistencia de datos, se combinan MongoDB y PostgreSQL, aprovechando las ventajas de los modelos no relacional y relacional.

Finalmente, en la parte DevOps, el uso de GitHub y Docker Compose facilita la colaboración, la contenedorización y el despliegue reproducible de toda la aplicación.

Modelo de datos




Cita
idCita int4 U
fechaCita date
hora time
tipoConsulta text N
notas text N
atendida bool
idUsuario int4
idPaciente int4
estado text N
HistorialPaciente
idHistorial int4 U
observaciones text N
fechaActualizacion date
idPaciente int4
idUsuario int4 N
idCita int4 N
RegistroAntropometrico
idMedicion int4 U
fechaMedicion date
peso numeric(6, 2) N
altura numeric(4, 2) N
imc numeric(5, 2) N
grasaCorporal numeric(5, 2) N
observaciones text N
idPaciente int4
idUsuario int4
Paciente
idPaciente int4 U
nombrePaciente text
apellidos text
fechaNac text
emailPaciente text U
telefono text U
estado text
motivoBaja text N
bajaPor int4 N
DietaHistorial
idPaciente int4
Dietaid int4
detalle text N
snapshot_data text N
registrado_en date
Documento
id int4 U
titulo text
etiquetas text N
tipo text N
fuente text
url_descarga timestamp
idUsuario int4
ruta_archivo text N
idioma text N
estado text N
fecha_subida date N
Usuario
idUsuario int4 U
nombreUsuario text U
email text U
telefono text U
passwordHash text
foto bytea N
rol text
nombre text N
apellidos text N
activo bool
debeActualizarPassword bool
ultimoCambioPassword date N
PacienteUsuario
UsuarioidUsuario int4
PacienteidPaciente int4
idPacienteUsuario int4
DietaAsignacion
id int4
idDieta int4
idPaciente int4
idUsuario int4
version int4
es_actual bool
fecha_asignacion date
notificado_en date N
comentario text N
Dieta
id int4 U
nombre text
objetivo text N
fechaInicioDieta date
fechaFinDieta date N
idUsuario int4
idPaciente int4
descripcion text N
calorias_objetivo numeric(19, 0)
es_plantilla bool
estado text
publicada_en date
version int4
notas text N
idDieta int4
Conversacion
id int4
participantes text
claveParticipantes text
noLeidos text
ultimoMensaje text
creadoEn date
actualizadoEn date
Receta
id int4 U
nombre text
descripcion text N
calorias_objetivo numeric(6, 2) N
instrucciones text N
Alimento
id int4 U U
nombre text
calorias numeric(6, 2) N
proteinas numeric(6, 2) N
grasas numeric(6, 2) N
carbohidratos numeric(6, 2) N
descripcion text N
fibra numeric(19, 0) N
unidad_base text
micronutrientes text N
RecetaAlimento
AlimentoidAlimento int4
RecetaidReceta int4
cantidad numeric(19, 0)
unidad text
observaciones text N
id int4
DietaReceta
RecetaidReceta int4
DietaidDieta int4
id int4
dia_semana text N
momento text N
orden text N
porciones int4 U
Mensaje
id int4 U
contenido text
enviadoEn timestamp
leidoEn bool
idUsuarioRemitente int4
idUsuarioDestinatario int4
adjuntos text
creadoEn date
actualizadoEn date
id2 int4
tiene
tiene
está registrado
crea
registra
registra
registra
es atendido por
sigue
sube
atiende
asigna
envía
recibe
incluye
pertenece
contiene
está presente



SPA o MPA



MPA

Bizihotz está desarrollada como una aplicación web multi-página (MPA) servida como un sitio estático. Cada sección de la aplicación corresponde a un archivo HTML distinto (agenda, pacientes, dietas, documentos, etc.) y la navegación entre ellas se realiza mediante enlaces normales, recargando la página en cada cambio de vista. Cada pantalla carga su propio archivo JavaScript, que se encarga de la lógica de esa vista y de realizar llamadas fetch al API Gateway en FastAPI para obtener o enviar datos. No se utiliza ningún framework SPA ni enrutamiento en el navegador, lo que simplifica el frontend y lo mantiene separado del backend.

Lecciones aprendidas



En este proyecto he ido más allá de una aplicación sencilla y he desarrollado una arquitectura completa basada en microservicios, similar a la que se utiliza en entornos profesionales. Cada microservicio tiene una responsabilidad clara y está desacoplado del resto, lo que demuestra que entiendo la importancia de diseñar sistemas escalables y mantenibles, y no solo que “funcionen”.

He trabajado con tecnologías distintas dentro del mismo proyecto, combinando Node.js y Python según el tipo de funcionalidad, y he aprendido a comunicar estos servicios mediante APIs REST centralizadas a través de un API Gateway. Esto me ha permitido entender cómo se estructuran aplicaciones distribuidas reales y cómo se gestiona el acceso a múltiples servicios desde un único punto de entrada.

También he tenido que tomar decisiones técnicas razonadas, como el uso de MongoDB y PostgreSQL según el tipo de datos que maneja cada microservicio, o la implementación de comunicación en tiempo real con Socket.IO, que añade complejidad respecto a una aplicación CRUD tradicional. Además, he trabajado con generación de documentos y validación de datos, abordando problemas más cercanos a un entorno real.

Por último, todo el sistema está contenedorizado con Docker y Docker Compose, lo que permite levantar la aplicación completa de forma reproducible y demuestra que he tenido en cuenta no solo el desarrollo, sino también el despliegue. El proyecto no se queda en una prueba académica, sino que es un sistema funcional, bien organizado y coherente, y por todo ello considero que refleja un nivel de trabajo que merece una buena valoración.

bizihotz





Email o nombre de usuario

Contraseña

Iniciar sesión



a
nutricionista

27/10/2025
14:44:58

Agenda

Nuevo paciente

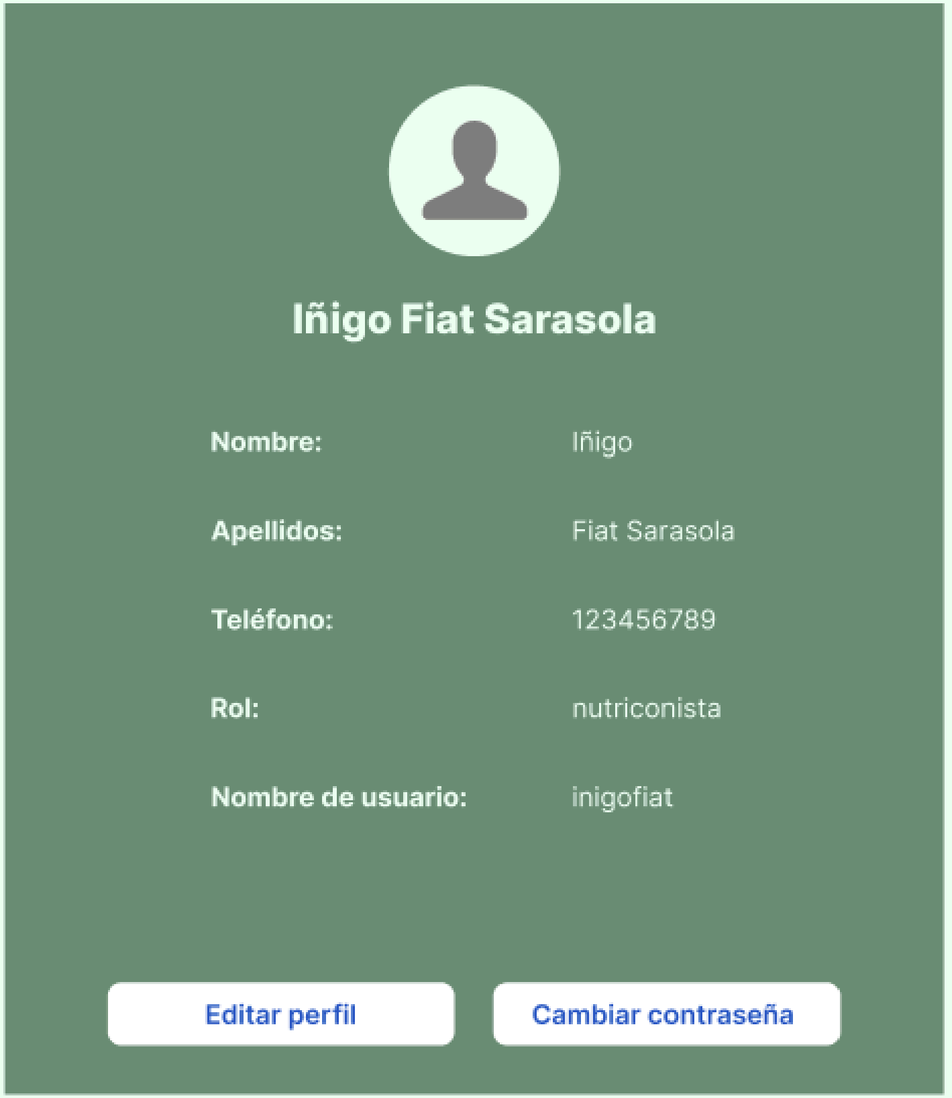
Mensajes

Documentos

Historial clínico

Dietas

HORA	PACIENTE	MOTIVO	ESTADO
09:00	Maddi Iturriaga Araneta	Valoracion inicial	Pendiente
09:45	Ekhi Aldanondo Zubizarreta	Seguimiento nutricional	Pendiente
10:30	Jon Goikoetxea Lizarralde	Reeducacion alimentaria	Pendiente
11:15	Ekhi Erro Muguruza	Seguimiento nutricional	Pendiente
12:00	Unai Mendizabal Arrizabalaga	Reeducacion alimentaria	Pendiente
12:45	Ainara Maroto Fernandez	ainara	Pendiente
15:00	June Erro Larrarte	Seguimiento nutricional	Pendiente
15:45	Amaia Otegi Arruabarrena	Consulta online	Pendiente
17:15	Haizea Etxaniz Araneta	Reeducacion alimentaria	Pendiente
18:00	Naroa Altuna Etxeberria	Consulta online	Pendiente


Iñigo Fiat Sarasola
Nombre:
Iñigo
Apellidos:
Fiat Sarasola
Teléfono:
123456789
Rol:
nutriconista
Nombre de usuario:
inigofiat
Editar perfil
Cambiar contraseña

Gestion de Calendarios y Citas

a | Hoy | ◀ | Octubre de 2025 | ▶ | Buscar paciente o motivo...

Poca carga · Media carga · Mucha carga

LUN	MAR	MIE	JUE	VIE	SAB	DOM
29	30	1 (10) 10 citas	2 (5) 5 citas	3 (2) 2 citas	4 Fin de semana	5 Fin de semana
6 (10) 10 citas	7 (3) 3 citas	8 (5) 5 citas	9 (10) 10 citas	10 (6) 6 citas	11 Fin de semana	12 Fin de semana
13 (2) 2 citas	14 (3) 3 citas	15 (2) 2 citas	16 (4) 4 citas	17 (2) 2 citas	18 Fin de semana	19 Fin de semana
20 (5) 5 citas	21 (11) 11 citas	22 (6) 6 citas	23 (10) 10 citas	24 (4) 4 citas	25 Fin de semana	26 Fin de semana
27 (10) 10 citas	28 (5) 5 citas	29 (3) 3 citas	30 (10) 10 citas	31 (5) 5 citas	1	2
3	4	5	6	7	8	9

Datos del paciente

Completa la información para incorporar a un nuevo paciente en la agenda.

Nombre

Apellidos

Fecha de nacimiento

dd/mm/aaaa

Email

Teléfono

Al guardar, el paciente quedará asociado a tu agenda de forma automática.

Guardar paciente

Cancelar

Lorea Idigoras Otegi

lorea.idigoras95@example.com · 609863041

39 años | Alta: 21/10/2025 | Actualizado: 21/10/2025

Datos de contacto

Nombre completo
Lorea Idigoras Otegi

Fecha de nacimiento
19/08/1986 (39 años)

Email
lorea.idigoras95@example.com

Teléfono
609863041

Última actualización
21 oct 2025, 14:57

Próximas citas

Ver agenda | Programar cita

Consulta online
28 oct 2025 - 10:30 Iñigo Fiat Pendiente

Seguimiento nutricional
29 oct 2025 - 18:00 Leire Altuna Pendiente

Seguimiento nutricional

Citas del paciente

164 citas

Fecha	Hora	Profesional	Motivo / Tipo	Estado	Notas
12 may 2026	18:00	a	Valoracion inicial	Pendiente	Consulta demo con Lorea Idigoras Otegi.

Conclusiones



Bizihotz es una aplicación completa que integra frontend, backend, bases de datos y despliegue en un único proyecto coherente. A lo largo del desarrollo se ha trabajado con tecnologías actuales y una arquitectura moderna, priorizando la separación de responsabilidades y la escalabilidad. El sistema es funcional, está bien estructurado y se puede ejecutar de forma sencilla, lo que demuestra una buena comprensión de conceptos clave como microservicios, APIs, gateway y contenedorización. En conjunto, el proyecto cumple los objetivos propuestos y refleja un enfoque práctico y realista del desarrollo de aplicaciones web.